



Análisis y Desarrollo de Software

Evidencia 4: Avances de la Base de Datos Proyecto Memorize Evolutivo

Modelo entidad-relación y almacenamiento persistente en PostgreSQL

Jose David Sandoval Caceres
Kevin Gregorio Mora

Ficha 3172526

SERVICIO NACIONAL DE APRENDIZAJE SENA
Atención al sector agropecuario
Piedecuesta - Santander

Este documento presenta la evidencia de la base de datos: el diagrama entidad-relacion (DER) del modelo de datos, las capturas del gestor de base de datos (Prisma Studio) mostrando las 7 tablas creadas con sus registros reales de prueba, y los scripts de creacion (DDL SQL generado por Prisma y el modelo del ORM en schema.prisma).

1. Modelo de datos: diagrama entidad-relacion

La base de datos es PostgreSQL 18 (base **memorize**), modelada con el ORM Prisma y normalizada: la tabla central **users** concentra identidad, progresion (level, xp), economia (coins, gems) y seguridad (role, baneos); las estadísticas y el inventario cuelgan en relacion 1:1; el historial de partidas y los logs administrativos en 1:N con borrado en cascada; y los anuncios y promociones son tablas de sistema sin relaciones.

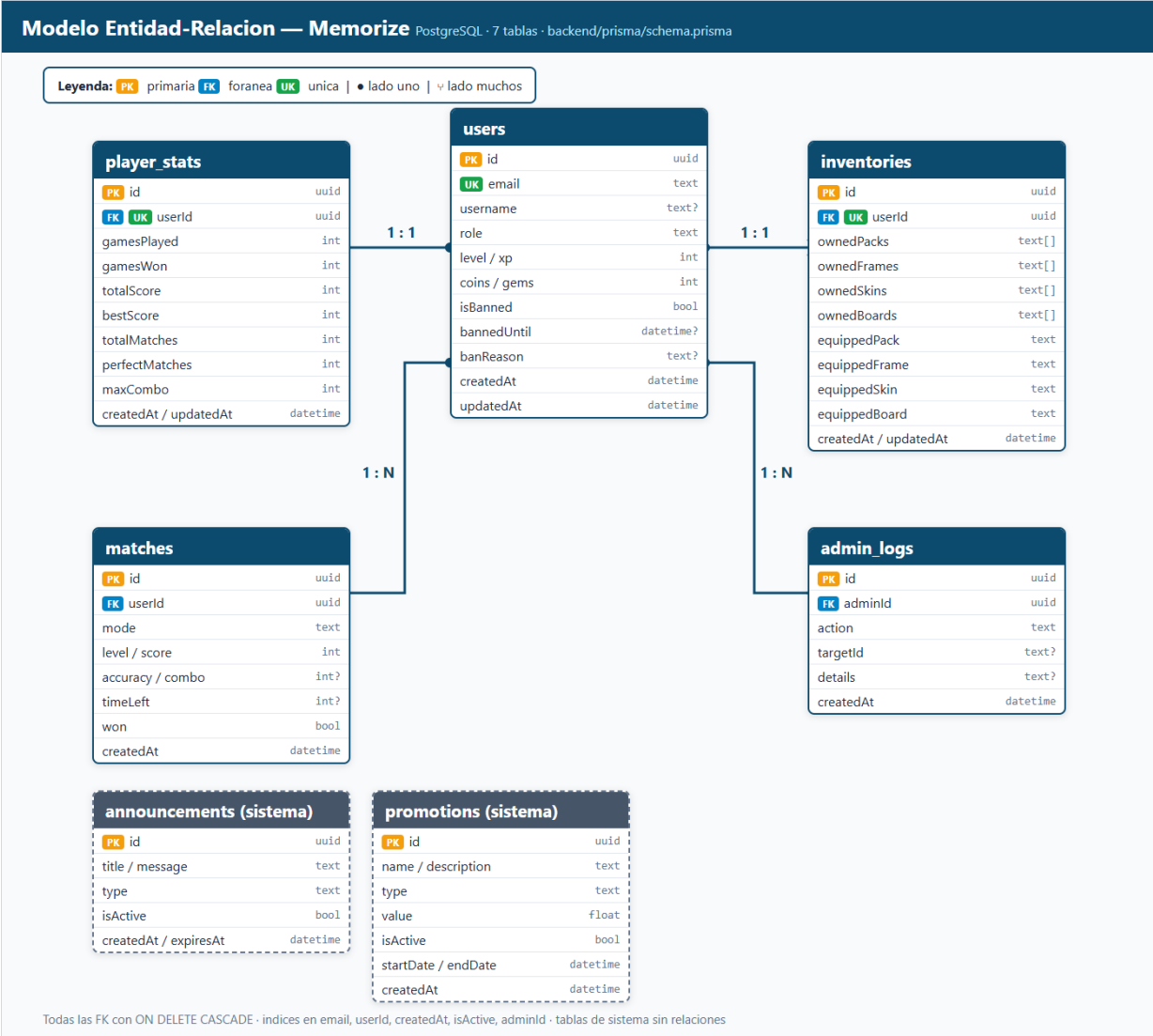


Figura 1 - Diagrama entidad-relacion (DER) de la base de datos.

Las 7 tablas con sus columnas, llaves primarias (PK), foraneas (FK) y unicas (UK), cardinalidades 1:1 y 1:N, e indices. Generado a partir del esquema real del proyecto.

Relaciones implementadas:

Tipo	Relacion	Implementacion
1 : 1	users - player_stats	player_stats.userId UNIQUE + FK a users.id

1 : 1	users - inventories	inventories.userId UNIQUE + FK a users.id
1 : N	users - matches	matches.userId FK a users.id (sin UNIQUE)
1 : N	users(admin) - admin_logs	admin_logs.adminId FK a users.id
Sin relacion	announcements, promotions	Tablas de sistema independientes

2. Evidencia de implementacion: tablas con registros reales

Capturas del gestor de base de datos **Prisma Studio** (<http://localhost:5555>, interfaz oficial del ORM del proyecto) conectado a la base **memorize** en PostgreSQL, mostrando las tablas creadas y sus registros de prueba generados por el backend y las APIs (login, partidas, acciones de administracion).

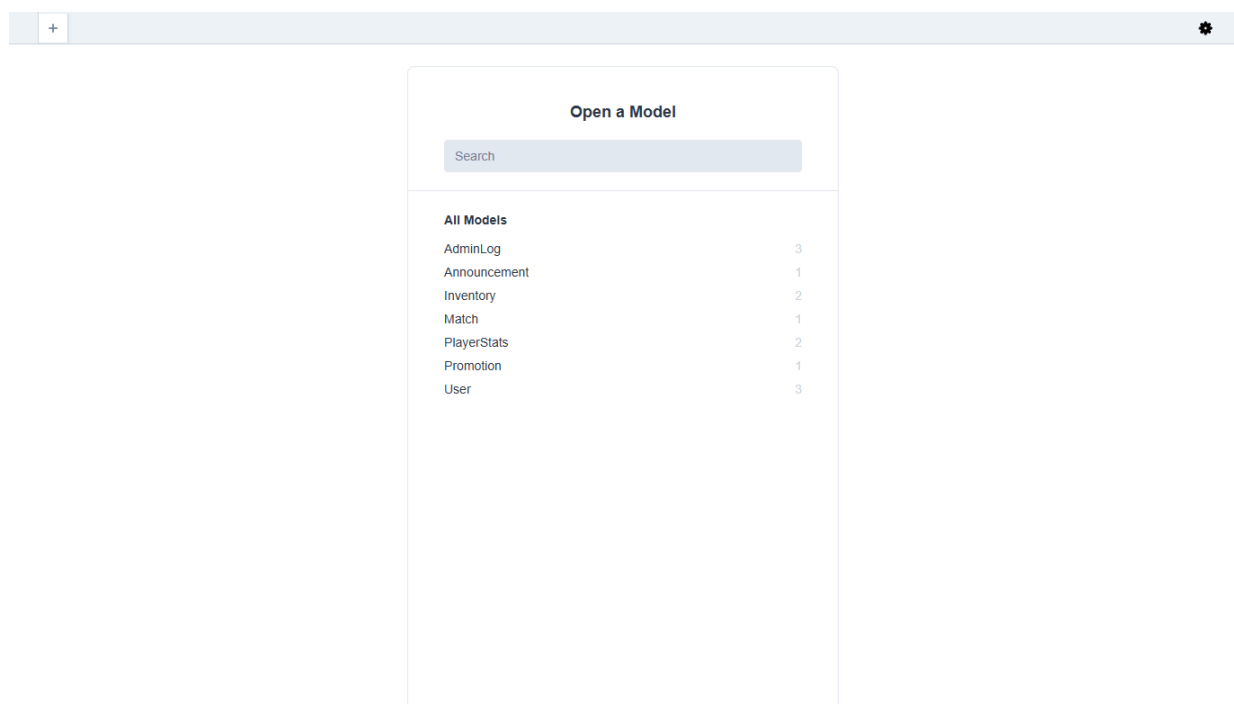


Figura 2 - Gestor: vista general de modelos.

Los 7 modelos con su conteo de registros: User (3), PlayerStats (2), Inventory (2), Match (1), Announcement (1), Promotion (1) y AdminLog (3).

User x + 						
	Filters	None	Fields	All	Showing 3 of 3	Add record
☐	id A	email A	username A?	role A	level #	coins #
☐	59f90123-c27d-4ba9-9160...	kevinmora2838@gmail.com	kevin	player	1	500
☐	6dbca2da-dcaf-4741-8efc...	rival.evidencia@univers...	rival.evidencia	player	1	1000
☐	e354b097-3b06-4240-9354...	evidencia.backend@unive...	evidencia.backend	admin	2	687

Figura 3 - Tabla users con registros.

Tres usuarios reales: rol (player/admin), nivel, XP, monedas y gemas persistidos por el servicio de autenticacion y las reglas de negocio.

Match x + 						
	Filters	None	Fields	All	Showing 1 of 1	Add record
☐	id A	userId A	user ()	mode A	level #?	accuracy #?
☐	b42a216d-691e-436d-a993...	e354b097-3b06-4240-9354...	User	classic	1	92

Figura 4 - Tabla matches con registros.

Historial de partidas (modo, nivel, puntaje, precision, combo, victoria) enlazado por llave foranea al usuario, con navegacion directa al registro padre.

Announcement × +							
	Filters None	Fields All	Showing 1 of 1	Add record			
☐	id ↕	title ↕	message ↕	type ↕	isActive ↕	createdAt 📅	expiresAt 📅?
☐	2580bd97-56f4-4b33-a9dd...	Bienvenidos a Memorize ...	Servidor en línea: jueg...	info	true	2026-10-05T16:01:47.880Z	null

Figura 7 - Tabla announcements con registros.
Anuncios globales del sistema creados desde el panel de administracion.

Promotion × +							
	Filters None	Fields All	Showing 1 of 1	Add record			
☐	id ↕	name ↕	description ↕	type ↕	value #	isActive ↕	startDate 📅
☐	b35c77be-fff0-4a73-874e...	Doble XP fin de semana	Gana el doble de XP en ...	coins_multiplier	2	true	2026-10-01T00:00:

Figura 8 - Tabla promotions con registros.
Promociones con tipo, valor multiplicador y vigencia por fechas.

AdminLog

+

FiltersNone

FieldsAll

Showing3 of 3

Add record

☐	id	adminId	admin	action	targetId	details	createdAt
☐	8f44722f-3d33-42d5-8d5c...	e354b097-3b06-4240-9354...	User	give_currency	6dbca2da-dcaf-4741-8efc...	{"coins":1000,"gems":10...	2026-10-05T14:50:
☐	90a0e519-18df-47c8-8446...	e354b097-3b06-4240-9354...	User	create_promotion	b35c77be-fff0-4a73-874e...	{"name":"Doble XP fin d...	2026-10-05T16:02:
☐	d6cf8cef-f617-4886-82a8...	e354b097-3b06-4240-9354...	User	create_announcement	2580bd97-56f4-4b33-a9dd...	{"title":"Bienvenidos a...	2026-10-05T16:01:

Figura 9 - Tabla admin_logs con registros.

Auditoria automatica: cada accion admin (dar moneda, crear promocion/anuncio) queda registrada con administrador, objetivo y detalle.

3.Codigo/Scripts: DDL SQL y modelo del ORM

Al usar el ORM Prisma, las tablas se definen en **backend/prisma/schema.prisma** y se aplican con **npx prisma db push**. El siguiente DDL es el script SQL real equivalente generado por Prisma para PostgreSQL (migrate diff del esquema):

Creacion de las tablas principales:

memorize_ddl.sql - CREATE TABLE users, player_stats (extracto real)

```
-- CreateTable
CREATE TABLE "users" (
  "id" TEXT NOT NULL,
  "email" TEXT NOT NULL,
  "username" TEXT,
  "role" TEXT NOT NULL DEFAULT 'player',
  "level" INTEGER NOT NULL DEFAULT 1,
  "xp" INTEGER NOT NULL DEFAULT 0,
  "coins" INTEGER NOT NULL DEFAULT 0,
  "gems" INTEGER NOT NULL DEFAULT 0,
  "isBanned" BOOLEAN NOT NULL DEFAULT false,
  "bannedUntil" TIMESTAMP(3),
  "banReason" TEXT,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL,
  CONSTRAINT "users_pkey" PRIMARY KEY ("id")
);
-- CreateTable
CREATE TABLE "player_stats" (
  "id" TEXT NOT NULL,
  "userId" TEXT NOT NULL,
  "gamesPlayed" INTEGER NOT NULL DEFAULT 0,
  ...
  CONSTRAINT "player_stats_pkey" PRIMARY KEY ("id")
);
```

Indices unicos (integridad 1:1 y busquedas) y llaves foraneas:

memorize_ddl.sql - indices y claves foraneas (extracto real)

```
-- CreateIndex
CREATE UNIQUE INDEX "users_email_key" ON "users"("email");
CREATE UNIQUE INDEX "player_stats_userId_key" ON "player_stats"("userId");
CREATE UNIQUE INDEX "inventories_userId_key" ON "inventories"("userId");
CREATE INDEX "matches_userId_idx" ON "matches"("userId");

-- AddForeignKey
ALTER TABLE "player_stats" ADD CONSTRAINT "player_stats_userId_fkey"
    FOREIGN KEY ("userId") REFERENCES "users"("id") ON DELETE CASCADE ON UPDATE CASCADE;
ALTER TABLE "matches" ADD CONSTRAINT "matches_userId_fkey"
    FOREIGN KEY ("userId") REFERENCES "users"("id") ON DELETE CASCADE ON UPDATE CASCADE;
ALTER TABLE "admin_logs" ADD CONSTRAINT "admin_logs_adminId_fkey"
    FOREIGN KEY ("adminId") REFERENCES "users"("id") ON DELETE CASCADE ON UPDATE CASCADE;
```

Modelo del ORM (fuente del DDL anterior):

backend/prisma/schema.prisma - modelo User y relaciones (extracto real)

```
model User {
  id      String    @id @default(uuid())
  email   String    @unique
  username String?
  role    String    @default("player") // "player" o "admin"
  level   Int       @default(1)
  xp      Int       @default(0)
  coins   Int       @default(0)
  gems    Int       @default(0)
  isBanned Boolean   @default(false)
  bannedUntil DateTime?
  banReason String?

  // Relaciones 1:1
  stats      PlayerStats? // Un usuario tiene una estadística
  inventory   Inventory?   // Un usuario tiene un inventario

  // Relaciones 1:N
  matches     Match[]      // Un usuario tiene muchas partidas
  adminLogs   AdminLog[]   // Un admin tiene muchos logs

  @@map("users")
}
```

4. Conclusion y cumplimiento

Criterio de la entrega	Estado
Modelo de datos (DER o esquema)	Cumple: Figura 1, diagrama ER de las 7 tablas con PK/FK/UK y cardinalidades
Gestor BD con tablas y registros de prueba	Cumple: Figuras 2-9, Prisma Studio sobre PostgreSQL con registros reales
Script SQL o modelos/migraciones del ORM	Cumple: DDL real (CREATE TABLE, índices, FK) + schema.prisma (sección 3)

La base de datos de Memorize Evolutivo está diseñada (modelo ER normalizado con relaciones 1:1 y 1:N), implementada (7 tablas en PostgreSQL con índices y borrado en cascada) y poblada con registros reales generados por el backend. El almacenamiento persistente, organizado y seguro del sistema está verificado en vivo.

Fin de la Evidencia 4 - 01 de octubre de 2026